

01 TELEMETRY RECEIVER Specification

- [SMC Telemetry Receiver — Low-Level Design](#)
 - [1. Module structure](#)
 - [1.1 Member-declaration order](#)
 - [2. TLM-2.0 interface](#)
 - [3. Register decode](#)
 - [4. Telemetry datapath](#)
 - [4.1 Assembly buffer](#)
 - [4.2 Message decode](#)
 - [4.3 Message queue](#)
 - [5. Processes and events](#)
 - [5.1 Single-driver discipline and output caching](#)
 - [6. Interrupt aggregation](#)
 - [7. Debug / introspection](#)
 - [8. Configuration \(CCI\)](#)
 - [9. ATB ingress back door](#)
 - [10. Known limitations](#)

SMC Telemetry Receiver — Low-Level Design

Implementation notes for the SystemC/TLM-2.0 LT model in `smc/peripherals/telemetry_receiver` (`include/telemetry_receiver.h`, `src/telemetry_receiver.cpp`). Read `01_TELEMETRY_RECEIVER_Specification.md` first for externally-observable behaviour.

1. Module structure

`smc::telemetry_receiver` is a single `SC_MODULE` with:

- one TLM-2.0 target socket (`reg_socket`, 32-bit AXI4-Lite-style),
- an active-low reset input (`rst_n_i`),
- the ATB flush-acknowledge input (`afready_i`),
- four outputs: `irq_o`, `afvalid_o`, `atready_o`, `debug_o` (4-bit),
- three CCI parameters (`buffer_depth`, `max_counters_per_message`, `access_delay_ns`).

It reuses the shared, framework-free register helpers in `common/include`:

- `regmodel::Register32` (`reg_access.h`) — a register with a fixed read/write-mask + reset contract, used for the three **storage** registers `CTRL`, `INTR_ENABLE`, `INTR_TEST`.
- `regmodel::RegisterMap32` (`reg_map.h`) — an offset → register dispatch table owning those three.
- `regmodel::apply_wlc` — the W1C arithmetic for `INTR_STATUS.MISSING_LAST`.

Everything else is **hardware-computed** and therefore not in the map: `STATUS` and `INTR_STATUS` are derived from state on each read, and `TELEMETRY_PROBE_ID` / `TELEMETRY_COUNTER_VLDS` / `TELEMETRY_COUNTER[i]` are a view of the oldest queued message. `CTRL` and `INTR_TEST` are in the map for their *storage* only; their write side effects live in `reg_write`.

1.1 Member-declaration order

The two immutable CCI parameters are declared **before** the ports and the sized containers, because their resolved values size `assembly_` and `queue_` in the constructor body. This is the ordering convention required by `.cursor/rules/cci-parameters.mdc`.

2. TLM-2.0 interface

`b_transport` is loosely-timed and **never calls** `wait()`. It:

1. rejects non-read/write commands → `TLM_COMMAND_ERROR_RESPONSE`;
2. requires `data_length == 4` (`accesswidth = 32` in the RDL) → else `TLM_BURST_ERROR_RESPONSE`;
3. requires `addr < 0x100` and 4-byte alignment → else `TLM_ADDRESS_ERROR_RESPONSE`;
4. dispatches to `reg_read` / `reg_write`; an in-window offset that decodes to no register (the `[0x1C, 0x80)` gap) also returns `TLM_ADDRESS_ERROR_RESPONSE`;
5. adds `access_delay_ns` to the annotated `delay` and sets `dmi_allowed = false`.

`transport_dbg` performs the same decode with **no** timing and no side effects (reads go through `dbg_reg`), returning the number of bytes transferred (4) or 0 when the access is malformed.

Temporal decoupling (quantum keeper) is the initiator's responsibility; the test-bench driver owns a `tlm_utils::tlm_quantumkeeper`.

3. Register decode

```
reg_read(off):
  STATUS          -> derive from count_ (empty / full)
  INTR_STATUS      -> missing_last_status_ | threshold_irq_active()
  TELEMETRY_PROBE_ID -> visible_message().probe_id & 0x1F
  TELEMETRY_COUNTER_VLDS -> OR of per-counter vld bits of visible_message()
  TELEMETRY_COUNTER[i] -> visible_message().counters[i].value
                      (0 when i >= max_counters_per_message)
  else             -> regmap_.read(off)  (CTRL / INTR_ENABLE / INTR_TEST)
  miss             -> false  (=> ADDRESS_ERROR)

reg_write(off, data):
  CTRL            -> RX_FLUSH ? rx_flush() : BUFFER_POP ? pop_message() : -
                      regmap_.write(off, data)      # stores TX_FLUSH + THRESHOLD
                      if TX_FLUSH set && afready_i    -> af_event_.notify()
                      recompute
  INTR_STATUS      -> apply_wlc(cur, data, MISSING_LAST); recompute
  INTR_ENABLE      -> regmap_.write(off, data);          recompute
  INTR_TEST        -> regmap_.write(off, data)
                      if data.MISSING_LAST -> set_missing_last_status() # pulse
                      recompute
  STATUS / PROBE_ID / COUNTER_VLDS / COUNTER[i] -> ignored (R0, write-ignore)
  miss            -> false  (=> ADDRESS_ERROR)
```

The `Register32` write-masks enforce the RDL field layout, so reserved bits never store:

Register	Store mask	Effect
CTRL	TX_FLUSH \ THRESHOLD (0x00FF_F100)	The two write-pulse bits (BUFFER_POP, RX_FLUSH) are consumed by <code>reg_write</code> and read back as 0.
INTR_ENABLE	INTR_MASK (0x11)	Reserved bits read 0.
INTR_TEST	INTR_BUFFER_THRESHOLD (0x10)	Only the level bit stores; MISSING_LAST is a pulse.

`counter_index(off)` maps an offset to a counter index and returns `-1` for offsets below `TELEMETRY_COUNTER0` or beyond the 32-entry array, so a stray offset can never index out of range (`dbg_reg` can be called directly with an arbitrary offset).

4. Telemetry datapath

4.1 Assembly buffer

`assembly_` is a byte vector sized once in the constructor:

```
assembly_.assign(telemetry_packets_per_message(max_counters_per_message)
                * BEATS_PER_PACKET, 0u);
```

`beats_` is the write cursor. `push_atb_beat()` stores one beat and only does work on packet boundaries:

```
assembly_[beats_++] = beat;
if (beats_ % 8 == 0) { // a packet just completed
    pkt = beats_ / 8 - 1;
    if (telemetry_packet_last(packet_word(pkt))) {
        queue_message(decode_message()); // end of message
        beats_ = 0;
    } else if (beats_ == assembly_.size()) {
        dbg_assembly_full_ = true; // no marker anywhere
        raise_missing_last();
        beats_ = 0; // discard partial message
    }
}
schedule_recompute(); // fill level may have moved
```

`packet_word(i)` re-assembles packet i from its eight beats — beat j carries packet bits $[8j+7 : 8j]$, so the first beat is the packet LSB.

4.2 Message decode

`decode_message()` walks blocks with a `(pkt, blk)` cursor that starts at `(0, 1)` — `blocks[0]` of packet 0 is the header — and wraps to the next packet after `blocks[6]`:

```
msg.probe_id = telemetry_packet_probe_id(packet_word(0)); // packet[60:56]
for each counter i:
    vld = true; val = 0;
    for j = 3 down to 0: // MSB byte first
        b = telemetry_packet_block(packet_word(pkt), blk);
        vld = vld && b.vld; // all 4 must be valid
        val |= b.data << (8 * j);
        advance (pkt, blk) // wraps at blk == 6
    msg.counters[i] = { vld, vld ? val : 0 }; // RTL zeroes invalid
```

The bit-field accessors (`telemetry_packet_last`, `telemetry_packet_probe_id`, `telemetry_packet_block`, `telemetry_set_packet_block`) are `constexpr` free functions in the header, so the packing rules exist in exactly one place and are usable at compile time.

4.3 Message queue

`queue_` is a `std::vector<telemetry_message>` of `buffer_depth` entries used as a circular buffer with a read index and a count (`rd_idx_`, `count_`) — no separate write pointer, so “full” and “empty” are unambiguous:

```

queue_message(msg):
    if (count_ == buffer_depth) {           // drop OLDEST, keep newest telemetry
        rd_idx_ = (rd_idx_ + 1) % buffer_depth;
        --count_;
    }
    queue_[(rd_idx_ + count_) % buffer_depth] = msg;
    ++count_;

pop_message():
    if (count_ == 0) return;                // RTL: pop only effective when non-empty
    rd_idx_ = (rd_idx_ + 1) % buffer_depth;
    --count_;

```

`visible_message()` returns `queue_[rd_idx_]`, or a static all-zero `empty_message_` when `count_ == 0`, which is what makes the whole software view read 0 on an empty queue.

`rx_flush()` resets `beats_`, `rd_idx_`, `count_` and the two latched debug flags in one place; it is used by both `CTRL.TELEMETRY_RX_FLUSH` and `reset`.

5. Processes and events

Three `SC_METHOD`s, all `dont_initialize()`:

Process	Sensitivity	Role
<code>reset_proc</code>	<code>rst_n_i</code>	On the low level: reset the three storage registers, clear <code>missing_last_status_</code> , call <code>rx_flush()</code> , and drop <code>accepting_</code> (so <code>atready_o</code> de-asserts and beats are refused). On the high level: re-assert <code>accepting_</code> .
<code>af_handshake_method</code>	<code>afready_i</code> , <code>af_event_</code>	Models <code>CTRL.TELEMETRY_TX_FLUSH.hwclr = afready_i && afvalid_o</code> : clears the stored <code>TX_FLUSH</code> bit when the transmitter acknowledges.
<code>recompute_method</code>	<code>recompute_event_</code>	The sole driver of all four outputs.

`start_of_simulation()` schedules one `recompute` so every output is defined before any stimulus arrives.

Every state-changing path (`reg_write`, `push_atb_beat`, `reset_proc`, `af_handshake_method`) calls `schedule_recompute()`, which is `recompute_event_.notify(SC_ZERO_TIME)`.

`af_event_` exists for one corner case: if software writes `TX_FLUSH` while `afready_i` is **already** high, there is no `afready_i` edge to trigger the handshake process, so `reg_write` notifies `af_event_` explicitly and the flush retires in the same delta cycle.

5.1 Single-driver discipline and output caching

`recompute_method` is the only writer of `irq_o`, `afvalid_o`, `atready_o` and `debug_o`. It computes all four values, then writes each `sc_signal` **only when the value changed** (guarded by `out_*_` shadow copies plus

an `outputs_valid_` flag that forces the first drive). This keeps the delta-cycle count low and matches the pattern used by `uart`, `plic`, and `beu`.

6. Interrupt aggregation

```
irq = missing_last_status_ || threshold_irq_active();

threshold_compare_value() = (CTRL.BUFFER_THRESHOLD) & threshold_wrap_mask_;
threshold_irq_active()    = ((count_ > threshold_compare_value())
                             || INTR_TEST.BUFFER_THRESHOLD)
                             && INTR_ENABLE.BUFFER_THRESHOLD;
```

`threshold_wrap_mask_` is computed once in the constructor as $(1 \ll (\text{clog2}(\text{buffer_depth}) + 1)) - 1$, reproducing the RTL's cast of the 12-bit `CTRL.BUFFER_THRESHOLD` field down to the message-buffer pointer width.

The two sources differ in kind, and that difference is deliberate:

- **Threshold** is recomputed from live state on every read of `INTR_STATUS`, so the status bit is a pure read-only mirror — a W1C write cannot clear it.
- **Missing-last** is stored in `missing_last_status_`. `set_missing_last_status()` applies the RTL's enable gate **at event time** (`if (INTR_ENABLE.MISSING_LAST) missing_last_status_ = true;`), so an event that arrives while masked is lost rather than deferred. `raise_missing_last()` additionally sets `dbg_missing_last_`, which is *not* enable-gated — the event stays observable on `debug_o` even when the interrupt is masked.

7. Debug / introspection

- `dbg_reg(off)` — side-effect-free peek used by `transport_dbg` and tests; returns 0 for any offset that does not decode.
- `dump_state(os)` — prints the resolved CCI parameters, the storage registers, queue geometry (fill / read index / threshold), assembly-buffer progress, the output states, and the currently visible message with per-counter valid bits.
- `fill_level()` — queue occupancy, for test benches.
- `push_atb_beat()` / `push_atb_beats()` — the ATB ingress back door (§9).

Logging uses `common/include/sim_log.h`: `SIM_LOG_INFO` for the one-shot CCI config summary, `SIM_LOG_TRACE` for per-transaction register access and queue push/pop, and `SIM_LOG_DEBUG` for decode misses, dropped beats, an ignored pop, buffer overflow, and the missing-last event.

8. Configuration (CCI)

Parameter	Type	Default	Mutability	Notes
buffer_depth	cci_param<unsigned, CCI_IMMUTABLE_PARAM>	8	immutable	Sizes queue_ and threshold_wrap_mask_. Must be ≥ 2.
max_counters_per_message	cci_param<unsigned, CCI_IMMUTABLE_PARAM>	4	immutable	Sizes assembly_ and each message's counter vector. Must be 1..32.
access_delay_ns	cci_param<double>	2.0	mutable	Annotated b_transport delay; re-read on every access, carries unit = nanoseconds metadata.

Both sizing parameters are **immutable**: they determine container sizes at elaboration and cannot change afterwards. They are validated in the constructor and violations raise `SC_REPORT_FATAL` (matching the RTL's `paramCheckBufferDepth` assertion). The constructor logs the resolved values, their provenance (`[preset]` / `[default]`), the derived assembly-buffer size, and the threshold wrap mask.

Presets use the full hierarchical name:

```
broker.set_preset_cci_value("top.telemetry0.buffer_depth", cci::cci_value(16u));
broker.set_preset_cci_value("top.telemetry0.max_counters_per_message",
                           cci::cci_value(8u));
```

9. ATB ingress back door

There is no ATB initiator in the model, so beats are injected directly:

```
bool      push_atb_beat (uint8_t beat);           // false if already low
unsigned  push_atb_beats(const std::vector<uint8_t>& beats); // beats accepted
```

`telemetry_encode_message(probe_id, counters, max_counters, set_last_packet)` is the reference **encoder** — the inverse of `decode_message()` — and produces a conforming beat stream. It lives in the model's header rather than in a test helper for two reasons: it keeps the packing rules in one place next to the decoder, and a future platform-level ATB bridge (or a transmitter model) can use it directly. Passing `set_last_packet = false` produces a malformed message, which is how the missing-last path is provoked in tests.

10. Known limitations

- **No ATB port.** Ingress is the back door above; `atready_o` reflects reset only, and ATB back-pressure/throughput is not modeled. Adding a real ATB target port is additive — the decode path already consumes beats one at a time.
- **No transmitter model.** `afvalid_o` / `afready_i` implement the flush handshake, but nothing produces telemetry autonomously.
- **No cycle-level timing.** A single `access_delay_ns` scalar approximates AXI4-Lite register latency; `b_transport` never blocks.

- **Flush/pulse artefacts.** The RTL's one-cycle `atready_o` dip during a flush and its single-cycle `debug_o` pulses have no cycle to occupy in an LT model; the pulses are latched until RX flush or reset so they stay checkable.
- **The instance base address is not modeled** — the platform's address router supplies it. See `01_TELEMETRY_RECEIVER_Specification.md` §2.1 for the unresolved `0xC000_9000` vs `0xC000_D000` map conflict.